

Подготовка данных для ИИ-отчёта клиента — чек-лист и баг-репорты

Описание задачи

- **Цель:** реализовать корректный отчёт с использованием ИИ на основе данных о посещениях и транзакциях клиента.
- **Ожидалось:** отчёт должен соответствовать заданному промту — содержать саммари и n пунктов анализа с точными ссылками на данные.
- **Что тестировалось:** бизнес-логика формирования отчёта, корректность расчётов по данным API, сопоставление отчёта с реальными данными.

Окружение

- Платформа: <platform>
- API: <base_url>/<endpoint>
- Токены: <user_token>
- Данные: <visits_history>, <lessons_history>, <deposit_details>

Чек-лист несоответствий (сопоставление отчёта и фактов)

№	Раздел отчёта	Что указано в отчёте	Фактические данные	Связанные материалы
1	Саммари / посещения	<n> посещений за год	<m> посещений за период	приложен скрипт для подсчёта (см. Приложение А)
2	Рейтинг по дням недели	все <n> посещений в один день (например, понедельник)	распределение: пн <counter1>, вт <counter2>, ср <counter3>, чт <counter4>, пт <counter5>, сб <counter6>, вс <counter7>	приложен скрипт для подсчёта (см. Приложение А)
3	Рейтинг по времени суток	утро <n1>, день <n2>, вечер <n3>	утро <m1>, день <m2>, вечер <m3>	 отражено в баг-репорте 2; приложен скрипт для подсчёта (см. Приложение А)
4	Рейтинг по неделям	данные по неделям: <n1>, <n2>, <n3> ...	фактические: неделя <w1> — <count1> посещений, неделя <w2> — <count2>, ...	
5	Риск оттока	низкий (0 дней)	прошло > <days> дней → высокий риск	 отражено в баг-репорте 1; приложен скрипт для подсчёта (см. Приложение А)
6	Тренировки/тренеры	занятия в один день, один тренер	занятия в разные дни, тренеры <Trainer1>, <Trainer2>, <Trainer3>	
7	Общая сумма покупок	<sum1>	<sum2>	

№	Раздел отчёта	Что указано в отчёте	Фактические данные	Связанные материалы
8	Средний чек	<avg1>	<avg2>	
9	Кол-во транзакций/депозитов	<t1> / <d1>	<t2> / <d2>	
10	Топ категории	депозиты <x1>, категория А <x2>, категория В <x3>	депозиты <y1>, категория А <y2>, категория В <y3>	
11	Кросс-продажи	предложены бьюти-услуги на основе истории	бьюти-услуги встречаются в старых транзакциях, в актуальных данных отсутствуют	⚠ отражено как «замечание по улучшению»

Чек-лист проверок

№	Проверка	Источник данных (API/объект)
1	Проверка структуры отчёта: наличие саммари и ровно 13 аналитических строк по промпту	Формат отчёта (response body)
2	Проверка стиля: отсутствие Markdown/списков/заголовков, формулировки ссылаются на данные	Формат отчёта
3	При полном отсутствии данных (нет посещений/покупок) — выводится «Недостаточно данных для анализа»	<visits_history>, <deposit_details>
4	При малом объёме (1–2 посещения или транзакции) — также «Недостаточно данных для анализа»	<visits_history>, <deposit_details>
5	Корректность подсчёта общего количества посещений	<visits_history>
6	Корректность распределения по неделям	<visits_history>
7	Корректность распределения по дням недели	<visits_history>
8	Корректность распределения по времени суток (утро/день/вечер)	<visits_history>
9	Корректность расчёта риска оттока по последнему посещению	<visits_history>
10	Корректность подсчёта общей суммы покупок (учёт по модулю, обработка возвратов)	<deposit_details>.transactions
11	Корректность расчёта среднего чека (округление до целого, в т.ч. при отрицательных значениях)	<deposit_details>.transactions
12	Корректность определения частоты повторных покупок (низкая/умеренная/высокая)	<deposit_details>.transactions
13	Корректность определения топ-категорий покупок	<deposit_details>.transactions
14	Корректное определение предпочтений клиента (время, дни, категории, тренеры)	<visits_history>, <lessons_history>
15	Правильность выявления тенденций (рост/снижение/стабильность) согласно правилу «3 из 4»	<visits_history>, <deposit_details>
16	Граничные значения риска оттока: 6, 7, 14, 15 дней	<visits_history>
17	Граничные значения частоты покупок: 2/3/5 покупок	<deposit_details>.transactions
18	Корректность определения тренда в пограничных случаях (рост/спад в 2 из 4 отрезков)	<visits_history>, <deposit_details>

№	Проверка	Источник данных (API/объект)
19	Валидность и точность рекомендаций по удержанию (основаны на динамике, не субъективны)	<visits_history>, <deposit_details>
20	Валидность рекомендаций по продлению/кросс-продажам (основаны на актуальных данных, не на устаревших)	<deposit_details>.transactions, <visits_history>

Баг-репорты (2 ключевых)



Баг-репорт 1. Некорректный расчёт риска оттока

Окружение: <base_url>, роль <client_role>

Шаги воспроизведения:

1. Получить историю посещений: GET <base_url>/get_client_visit_history.
2. Зафиксировать дату последнего посещения из ответа API.
3. Сформировать отчёт и сравнить поле «Риск оттока».

ER: категория риска рассчитывается по правилам; отображается корректное число дней.

AR: в отчёте «низкий — 0 дней назад», при фактическом интервале <days> дней (на момент формирования отчёта).

Статус: Critical — ломает основную бизнес-логику

Приложения (рекомендуемые):

- копия запроса и ответа API (для воспроизведения шага 1),
- скриншот блока отчёта (для AR),
- скрипт Postman для расчёта посещений (см. Приложение А).



Баг-репорт 2. Искажение распределения посещений по времени суток (пункт №3 чек-листа)

Окружение: <base_url>, роль <client_role>

Шаги воспроизведения:

1. Получить GET get_client_visit_history.
2. Для каждого посещения определить время начала и отнести к «утро/день/вечер» по принятой границе.
3. Сверить агрегаты с тем, что вывел отчёт.

ER: значения «утро/день/вечер» соответствуют расчёту из API.

AR: отчёт показывает завышенные значения днём/вечером и большее общее количество, чем в данных.

Статус: High — влияет на качество анализа, но не блокирует ключевую функцию

Приложения (рекомендуемые):

- копия запроса и ответа API (для воспроизведения шага 1),
- скрипт Postman для расчёта распределения по времени суток (см. Приложение А)
- сводная таблица распределения (для анализа на шаге 3),
- скриншот блока отчёта (для AR).



Дополнительные наблюдения / замечания по улучшению

(аналитическая часть тестирования — мой вклад в доработку требований)

- **Уточнение периода анализа в промпте** — сейчас отчёт захватывает устаревшие транзакции и бьюти-услуги. Я предложила добавить правило

«использовать последние N месяцев» и/или «старые данные выносить в блок *«Исторические факты»* с указанием периода».

- **Ссылки на источник данных** — в текущем виде отчёт часто ссылается на некорректные числа. Предложение: закрепить в промпте требование «каждый вывод = обязательная ссылка на количество и даты из API».
- **Единообразие расчёта недель** — сейчас смешиваются ISO-недели и недели месяца. Моё предложение: явно зафиксировать метод (ISO-стандарт или «недели месяца») и использовать его в коде/промпте.

Приложение А. Postman Tests: расчёт распределений и контроль целостности

Назначение: Данный скрипт использовался для получения фактических данных, указанных в чек-листе и баг-репортах 1–2: для автоматической проверки корректности распределения посещений клиента по времени суток и дням недели, а также для расчёта интервала с последнего посещения.

Контекст запуска: Postman → вкладка Tests (выполняется после ответа API)

Функциональность скрипта:

- Подсчёт общего количества посещений.
- Распределение посещений по времени суток (утро, день, вечер).
- Распределение посещений по дням недели (пн–вс).
- Расчёт дней, прошедших с последнего посещения.
- Автоматические проверки (pm.test) на совпадение суммы подсчётов с общим числом посещений.

Код:

```
try {
  //Парсим ответ и берём историю посещений.
  var bodyRes = pm.response.json();

  //Получаем общее количество посещений
  var histArr = bodyRes.visits_history;
  console.log("посещений: --> " + histArr.length);

  //Счётчики по времени суток:
  var morning = 0;
  var afternoon = 0;
  var evening = 0;

  //Счётчики по дням недели
  var sunday = 0;
  var monday = 0;
  var tuesday = 0;
  var wednesday = 0;
  var thursday = 0;
  var friday = 0;
  var saturday = 0;

  //Проход по всем посещениям
  for (var i = 0; i < histArr.length; i++) {

    //Получить количество посещений по времени суток:
    //утро: 08:00–11:59, день: 12:00–16:59, вечер: всё остальное (включая «ночь»)
```

```

var time = parseInt(histArr[i].name.split(":")[0].trim(), 10);
if (time >= 8 && time < 12) {
    morning++;
}
else if (time >= 12 && time < 17) {
    afternoon++;
}
else {
    evening++;
}

//Получить количество дней с последнего посещения
var date = new Date(histArr[i].date);
if (i === 0) {
    var today = new Date();
    var diffDays = Math.floor((today - date) / (1000 * 60 * 60 * 24));
    console.log("Последний день посещения: " + date + "; Прошло дней: " + diffDays);
}

//Получить количество посещений по дням недели:
//Получить день недели
//0 → "Sunday", 1 → "Monday", 2 → "Tuesday", 3 → "Wednesday", 4 → "Thursday", 5 → "Friday", 6 →
"Saturday"
switch (date.getDay()) {
    case 0: sunday++; break;
    case 1: monday++; break;
    case 2: tuesday++; break;
    case 3: wednesday++; break;
    case 4: thursday++; break;
    case 5: friday++; break;
    case 6: saturday++; break;
}
}

//Вывод результатов
console.log("утро: " + morning);
console.log("день: " + afternoon);
console.log("вечер: " + evening);
console.log("воскресенье: " + sunday);
console.log("понедельник: " + monday);
console.log("вторник: " + tuesday);
console.log("среда: " + wednesday);
console.log("четверг: " + thursday);
console.log("пятница: " + friday);
console.log("суббота: " + saturday);

//Авто-проверка целостности
pm.test("Сумма (утро+день+вечер) == числу посещений", function () {
    pm.expect(morning + afternoon + evening).to.eql(histArr.length);
});
pm.test("Сумма по дням недели == числу посещений", function () {
    pm.expect(sunday + monday + tuesday + wednesday + thursday + friday + saturday)
        .to.eql(histArr.length);
});

```

```
}  
catch (e) {  
    console.error(e);  
}
```